Predictively Oriented Posteriors in PyMC

Google Summer of Code 2026 — Project Proposal

Applicant	Advay Bhagwat
Email	advay.bhagwat@gmail.com
GitHub	eclipse1605
Institution	Indian Institute of Information Technology and Management, Gwalior (IIITM)
Degree	B.Tech. Computer Science & Engineering, 3rd Year
Timezone	IST (UTC+5:30)

Organisation	NumFOCUS / PyMC
Project	Predictively Oriented Posteriors
Duration	350 hours (Hard)
Mentors	Osvaldo Martin, Chris Fonnesbeck, Yann McLatchie

About Me

I am Advay Bhagwat, a third-year B.Tech. Computer Science and Engineering student at ABV Indian Institute of Information Technology and Management, Gwalior (IIITM), India. I have been contributing to PyMC and PyTensor since December 2025, with four pull requests merged and five more under active review. My work spans PyMC's logprob inference infrastructure (measurable graph rewrites, logp derivations for transforms and arithmetic operations), PyMC's tuning-statistics infrastructure (`sample_stats` tracking across step methods), and PyTensor's Scan and validation layer, building the PyTensor graph-manipulation fluency and familiarity with `model.compile_logp()` and the sampling-loop internals that the PrO sampler directly requires.

I am applying for GSoC 2026 to implement Predictively Oriented (PrO) posteriors in PyMC: a theoretically principled sampler that is provably superior to Gibbs/Bayes posteriors under model misspecification, yet reduces to the standard posterior in well-specified settings. The full algorithm is worked out in McLatchie et al. (2026) and the PyMC codebase is familiar territory to me; I am confident the 350-hour scope is achievable within the proposed timeline.

Abstract

Standard and generalised Bayesian posteriors collapse to a point mass as $n \to \infty$, which degrades predictive quality under model misspecification. *Predictively Oriented (PrO) posteriors* (McLatchie et al., 2026) rectify this by targeting uncertainty that is optimal for prediction, expressing parameter uncertainty as a direct consequence of predictive ability. Under non-trivial model misspecification, PrO posteriors are provably superior to Gibbs posteriors in predictive divergence; under correct specification they match them. Sampling from PrO posteriors is achieved by evolving interacting particles via mean-field Langevin dynamics derived from a Wasserstein gradient flow.

This proposal describes the design and implementation of `pymc-pro`, a standalone library (hosted at `pymc-devs/pymc-pro`) that exposes PrO posterior sampling for arbitrary PyMC models through a clean, `pm.sample`-like public API, together with a thin Bambi wrapper (`bambinos/bambi-pro`).

The core contribution spans: (i) a scoring-rule abstraction interfacing with PyMC's log-probability infrastructure and PyTensor autodiff; (ii) a PyTensor-native implementation of the Wasserstein gradient for both the logarithmic and the squared-MMD scoring rules; (iii) a vectorised Euler–Maruyama particle simulation loop; and (iv) ArviZ `DataTree` output, diagnostics, and tutorial notebooks illustrating robustness to model misspecification on real datasets.

Synopsis and Community Benefit

Motivation

Bayesian inference sits at the heart of PyMC. Yet, as McLatchie et al. (2026) detail, the classical Bayes posterior Π_n and its generalised extensions (Gibbs posteriors) share a structural limitation: they concentrate onto a point mass as $n \to \infty$ even when no single parameter $\theta^\star$ is sufficient to explain P_0. The resulting posterior predictive P_{Π_n} converges to a plug-in prediction, losing all distributional richness.

PRO posteriors resolve this by reorienting the variational objective from *best parameter* to *best predictive*:

$$Q_n \;=\; \operatorname*{argmin}_{Q \in \mathcal{P}(\Theta)} \left\{ \frac{\lambda_n}{n} \sum_{i=1}^{n} S(P_Q,\, x_i) + \mathrm{KL}(Q \,\|\, \Pi) \right\}, \tag{1}$$

where $P_Q := \int P_\theta \mathrm{d}Q(\theta)$ is the *predictive mixture* and S is any proper scoring rule (Gneiting & Raftery, 2007). This stands in sharp contrast to the Gibbs objective, which scores individual components P_θ rather than the mixture:

$$Q_n^\dagger \;=\; \operatorname*{argmin}_{Q \in \mathcal{P}(\Theta)} \left\{ \frac{\lambda_n}{n} \sum_{i=1}^{n} \int S(P_\theta, x_i)\, \mathrm{d}Q(\theta) + \mathrm{KL}(Q \,\|\, \Pi) \right\}. \tag{2}$$

In practical terms, this means that PyMC users working with generative models, time-series, spatial data, or any setting where the model family is an approximation will obtain posteriors that are *calibrated to predictive reality* rather than collapsing to overconfident point estimates.

Scientific Value

PRO posteriors make PyMC the first probabilistic programming language to offer a turnkey implementation of this novel inference paradigm. They broaden the ecosystem's appeal to practitioners dealing with misspecified models (an ubiquitous condition in applied statistics) and open new workflows for uncertainty quantification, model diagnostics, and predictive calibration assessments.

Technical Value

Implementing PRO posteriors within PyMC requires building a particle-based sampler that: (a) integrates tightly with PyTensor's autodiff to compute prior score gradients via `pytensor.grad`; (b) provides a new class of scoring-rule ops whose Wasserstein gradients drive particle evolution; and (c) feeds seamlessly into the existing ArviZ `DataTree` pipeline. This infrastructure is independently valuable: the scoring-rule abstraction can underpin future work on generalised Bayesian methods (e.g. power posteriors, β-divergence posteriors) already of interest to the community.

Mathematical Background

Proper Scoring Rules and Predictive Divergence

Let $S : \mathcal{P}(\mathcal{X}) \times \mathcal{X} \to \mathbb{R} \cup \{\infty\}$ be a proper scoring rule (Gneiting & Raftery, 2007), meaning

$$\mathcal{S}(P', P') \;\leq\; \mathcal{S}(P, P') \quad \forall\, P, P', \qquad \mathcal{S}(P, P') := \mathbb{E}_{X \sim P'}\big[S(P, X)\big].$$

The induced divergence $\mathcal{D}_S(P, P') := \mathcal{S}(P, P') - \mathcal{S}(P', P') \geq 0$ generalises the KL divergence (recovered when $S(P, x) = -\log p(x)$).

Two scoring rules are central to this work:

Log-score:
$S_{\log}(P, x) = -\log p(x)$, inducing $\mathcal{D}_{S_{\log}}(P, P_0) = \mathrm{KL}(P_0 \| P)$.

Squared MMD:
For a positive-definite kernel k, $S_{\mathrm{MMD}}(P, x) = \mathbb{E}_{X,X' \sim P}[k(X, X')] - 2\,\mathbb{E}_{X \sim P}[k(X, x)] + k(x, x)$, giving robustness to outliers and heavy tails (Chérief-Abdellatif & Alquier, 2020; Matsubara et al., 2022).

The PrO Posterior

With the notation $P_Q := \int P_\theta \mathsf{d}Q(\theta)$, the PRO posterior is the unique solution to (1).

An equivalent fixed-point characterisation is

$$\mathsf{d}Q_n(\theta) \;\propto\; \mathsf{d}\Pi(\theta) \cdot \exp\left\{ -\frac{\lambda_n}{n} \sum_{i=1}^{n} \frac{\delta S(P_Q, x_i)}{\delta Q}\bigg|_{Q=Q_n}(\theta) \right\}, \tag{3}$$

where $\delta/\delta Q$ denotes the functional derivative. Unlike the Gibbs posterior, (3) is *not* a coherent belief update (Bissiri et al., 2016): the exponent involves the derivative of $S(P_Q, \cdot)$ with respect to the *mixing measure* Q, not merely $\log p_\theta(x_i)$.

Key Theoretical Guarantees

We summarise the main results of McLatchie et al. (2026). In all statements, $\lambda_n = n^{1/2}/\sqrt{kC^2 \log(n)^2}$.

Predictive Domination (Thm. 1):
Under non-trivial misspecification (Definition 2 of ibid.), for n large enough but finite,

$$\mathbb{E}[\mathcal{D}_S(P_{Q_n}, P_0)] < \mathbb{E}[\mathcal{D}_S(P_{Q_n^\dagger}, P_0)],$$
$$\mathbb{E}[\mathcal{D}_S(P_{Q_n}, P_0) - \mathcal{D}_S(P_{Q^\star}, P_0)] \lesssim \nu_n,$$

where $\nu_n = O(\log(n)/\sqrt{n})$ and $Q^\star := \operatorname{arginf}_{Q \in \mathcal{P}(\Theta)} \mathcal{D}_S(P_Q, P_0)$.

Convex Recovery (Thm. 1, part CR):
When $P_0 = \int P_\theta \mathsf{d}Q^\star(\theta)$ for some $Q^\star \in \mathcal{P}(\Theta)$, the PRO posterior converges to P_0 at the parametric rate $n^{-1/2}$ (up to log factors).

Well-specification:
When $P_0 \in \mathcal{M}_\Theta = \{P_\theta : \theta \in \Theta\}$, the gap to the Gibbs posterior vanishes at rate ν_n, so there is essentially no cost to using Q_n in place of $Q_n^\dagger$.

These results hold simultaneously for the log-score and the MMD, making the algorithm scoring-rule agnostic in terms of its theoretical properties.

Computation: Wasserstein Gradient Flow and Mean-Field Langevin Dynamics

The Wasserstein Gradient Flow Functional

Rewriting (1), the objective is

$$\mathscr{L}(Q) \;=\; \underbrace{\frac{\lambda_n}{n}\sum_{i=1}^{n} S(P_Q, x_i)}_{\text{predictive loss}} \;-\; \int \log \mathrm{d}\Pi(\theta)\,\mathrm{d}Q(\theta) \;+\; \int \log \mathrm{d}Q(\theta)\,\mathrm{d}Q(\theta). \tag{4}$$

The Wasserstein gradient of (4) evaluated at $\vartheta \in \Theta$ is

$$\nabla_W \mathscr{L}(Q)[\vartheta] \;=\; \lambda_n\, \mathcal{W}(Q)[\vartheta] \;-\; \nabla_\vartheta \log \mathrm{d}\Pi(\vartheta) \;+\; \nabla_\vartheta \log \mathrm{d}Q(\vartheta), \tag{5}$$

where $\mathcal{W}(Q)[\vartheta] := \nabla_\vartheta\big[\frac{1}{n}\sum_i \frac{\delta S(P_Q, x_i)}{\delta Q}(\vartheta)\big]$ is the scoring-rule-specific interaction term.

Explicit forms for the two primary scoring rules:

$$\mathcal{W}_{\log}(Q)[\vartheta] \;=\; \frac{1}{n}\sum_{i=1}^{n} \frac{p_\vartheta(x_i)}{p_Q(x_i)}\,\nabla_\vartheta \log p_\vartheta(x_i), \tag{6}$$

$$\mathcal{W}_{\mathrm{MMD}}(Q)[\vartheta] \;=\; \frac{1}{n}\sum_{i=1}^{n} \int \nabla_1 \mathsf{L}_{\mathrm{MMD}}(\vartheta, \theta; x_i)\,\mathrm{d}Q(\theta). \tag{7}$$

Here $p_Q(x) := \int p_\theta(x)\,\mathrm{d}Q(\theta)$ is the predictive mixture density, and $\mathsf{L}_{\mathrm{MMD}}(\vartheta, \theta; x) := \langle \mu(P_\vartheta) - \mu(\delta_x),\, \mu(P_\theta) - \mu(\delta_x)\rangle_{\mathcal{H}}$ with ∇_1 denoting the gradient with respect to the first argument. Equation (6) follows from the functional derivative of the log-score (McLatchie et al., 2026, Appendix); equation (7) follows from Lemma 2 of ibid. (kernel-score decomposition) and the mean embedding $\mu_Q := \int \phi(\cdot)\mathrm{d}P_Q$.

McKean–Vlasov SDE and Particle Approximation

The Wasserstein gradient flow $\{Q_t\}_{t\geq 0}$ of (4) satisfies the McKean–Vlasov SDE

$$\mathrm{d}\vartheta_t \;=\; -\Big\{\lambda_n\, \mathcal{W}(Q_t)[\vartheta_t] - \nabla_\vartheta \log \mathrm{d}\Pi(\vartheta_t)\Big\}\mathrm{d}t \;+\; \sqrt{2}\,\mathrm{d}B_t, \qquad \vartheta_t \sim Q_t, \tag{8}$$

where B_t is standard Brownian motion and the entropy gradient $\nabla_W \mathrm{Ent}(Q) = \nabla_\vartheta \log q_t$ is absorbed into the diffusion term by Itô's formula (Jordan et al., 1998).

Since (8) depends on the unknown Q_t, we deploy an p-particle mean-field approximation $Q_t^{(j)} \approx \frac{1}{p}\sum_{\ell\neq j} \delta_{\vartheta_t^{(\ell)}}$, yielding the interacting particle system ($j = 1, \ldots, p$):

$$\mathrm{d}\vartheta_t^{(j)} \;=\; -\Big\{\mathcal{W}(Q_t^{(j)})[\vartheta_t^{(j)}] - \nabla_\vartheta \log \mathrm{d}\Pi(\vartheta_t^{(j)})\Big\}\mathrm{d}t \;+\; \sqrt{2}\,\mathrm{d}B_t^{(j)}, \tag{9}$$

where $\{B_t^{(j)}\}$ are independent Brownian motions.

Euler–Maruyama Discretisation

Discretising (9) with step size $\epsilon > 0$:

$$\vartheta_{(k+1)}^{(j)} \;=\; \vartheta_{(k)}^{(j)} \;-\; \epsilon\Big\{\mathcal{W}(\widehat{Q}_{(k)}^{(j)})[\vartheta_{(k)}^{(j)}] - \nabla_\vartheta \log \mathrm{d}\Pi(\vartheta_{(k)}^{(j)})\Big\} \;+\; \sqrt{2\epsilon}\,\xi_k^{(j)}, \quad \xi_k^{(j)} \sim \mathcal{N}(0, I_d). \tag{10}$$

After a burn-in of τ steps, the posterior is approximated by

$$Q_n \approx \frac{1}{|\{k : k > \tau\}|} \sum_{k>\tau} \widehat{Q}_{(k)}, \qquad \widehat{Q}_{(k)} = \frac{1}{p} \sum_{j=1}^{p} \delta_{\vartheta^{(j)}_{(k)}}. \tag{11}$$

Following the particle-limit argument used in McLatchie et al. (2026, Computation section), building on Wild et al. (2023, Appendix E, Lemma 6), and under the same regularity assumptions (including a Log-Sobolev condition for the target law, which yields exponential ergodicity of the mean-field Langevin dynamics), this estimator converges to Q_n as $p \to \infty$, $\epsilon \to 0$, and $K \to \infty$. Outside these assumptions, we do not claim a global exponential convergence guarantee.

Learning rate. Following McLatchie et al. (2026), the learning rate is

$$\lambda_n = \frac{n^{1/2}}{\sqrt{kC^2 \log(n)^2}},$$

with $k = 1$ under the entropy assumption and $k = \mathrm{ord}(S)$ under the global assumption. For usability, an `auto` mode in the API selects this value given n.

Computational Complexity and Mitigation Strategy

Let p be the number of particles, K the number of Euler–Maruyama steps, n the number of observations, and d the unconstrained parameter dimension.

For `LogScore`, each step requires per-observation log-likelihood and score evaluation across all particles; this is dominated by batched autodiff/model-evaluation cost and scales as $O(pn)$ model evaluations per step (plus vector operations), with memory dominated by $O(pn)$ tensors.

For `MMDScore`, the pairwise particle interaction term introduces an additional quadratic particle factor in the naive implementation; depending on the estimator, this is typically $O(p^2 n)$ (or $O(p^2 m)$ with m predictive samples) per step. Accordingly, total runtime can be substantial at paper-scale settings.

To keep wall-clock cost practical, the implementation plan includes:

1. **Required exact optimisations:** vectorised PyTensor graphs, backend compilation, and chunked evaluation over data/particle blocks to reduce memory pressure without changing the target.
2. **Required budget controls:** diagnostics-driven early stopping and explicit runtime guidance for (p, K) choices by model size.
3. **Optional approximate accelerations (clearly flagged):** mini-batch data updates and approximate MMD interactions (subsampled/blocked pairs, low-rank or random-feature kernel approximations), evaluated empirically before recommendation.

Approximate accelerations will be opt-in only and recorded in metadata; they will not be enabled silently.

Technical Implementation Plan

Packaging Structure

The work will live in two standalone repositories rather than inside the main PyMC codebase:

- `pymc-devs/pymc-pro` – the inference engine, with no Bambi dependency. Operates directly on any PyMC model.

- **bambinos/bambi-pro** – a thin Bambi wrapper that translates a Bambi model to the engine's interface by using `compile_forward_sampling_function` together with `DictToArrayBijection.rmap` from PyMC's internals to obtain model-agnostic predictive samples for any Bambi-specified likelihood.

```
# --- pymc-devs/pymc-pro ------------------------------------------
pymc_pro/                # importable package
    __init__.py          # public re-exports: sample_pro, LogScore, MMDScore
    scoring.py           # abstract ScoringRule + LogScore + MMDScore
    wgf.py               # Wasserstein gradient computation (PyTensor ops)
    particles.py         # ParticleState dataclass; EM update step
    sampler.py           # main simulation loop; burn-in; thinning
    diagnostics.py       # particle spread, ESS, convergence checks
    api.py               # sample_pro() -- user-facing entry point
tests/
    test_scoring.py
    test_wgf.py
    test_particles.py
    test_sampler.py
    test_api.py          # integration tests (Gaussian, logistic, GMM)
docs/
    notebooks/
        pro_tutorial.ipynb
pyproject.toml
README.md

# --- bambinos/bambi-pro ------------------------------------------
bambi_pro/               # importable package
    __init__.py
    model.py             # BambiPredictiveWrapper
    api.py               # sample_pro() thin wrapper for Bambi models
tests/
    test_model.py
    test_api.py
pyproject.toml
README.md
```

Listing 1: Repository layouts for `pymc-pro` and `bambi-pro`.

Scoring Rules (`scoring.py`)

An abstract base class decouples the scoring-rule mathematics from the particle loop, enabling future extensions (CRPS, energy score, etc.):

```
import abc
import pytensor.tensor as pt

class ScoringRule(abc.ABC):
    """Abstract proper scoring rule for P_Q scored against data x."""

    @abc.abstractmethod
    def wasserstein_gradient(
        self,
        particles: pt.TensorVariable,    # shape (p, d)
        data: pt.TensorVariable,          # shape (n, ...)
        logp_fn,                          # model.compile_logp()
    ) -> pt.TensorVariable:               # shape (p, d)
        """Compute W(Q^(j))[vartheta^(j)] for all j simultaneously."""
        ...

class LogScore(ScoringRule):
    """S(P, x) = -log p(x).  WGF = importance-weighted score."""

class MMDScore(ScoringRule):
```

```
    """S(P, x) = MMD^2(P, delta_x).  WGF = kernel mean embedding diff."""
    def __init__(self, kernel="rbf", bandwidth="median"):
        ...
```

Listing 2: Abstract scoring rule interface.

Log-score Wasserstein Gradient

From (6), the interaction term for particle j is the importance-weighted score function:

$$\mathcal{W}_{\log}^{(j)} = \frac{1}{n}\sum_{i=1}^{n} \frac{p_{\vartheta^{(j)}}(x_i)}{\hat{p}_Q^{(j)}(x_i)} \nabla_\vartheta \log p_{\vartheta^{(j)}}(x_i), \quad \hat{p}_Q^{(j)}(x_i) := \frac{1}{p-1}\sum_{\ell \neq j} p_{\vartheta^{(\ell)}}(x_i). \tag{12}$$

This requires per-observation likelihoods $p_\vartheta(x_i)$ for all particles and points, together with their gradients via `pytensor.grad`:

```
import pytensor
import pytensor.tensor as pt

def log_score_wgf_grad(particles, batched_obs_logp_fn, batched_obs_score_fn):
    # batched_obs_logp_fn(particles) -> (p, n) per-observation log-likelihoods
    # batched_obs_score_fn(particles) -> (p, n, d) per-observation score vectors
    all_obs_logp = batched_obs_logp_fn(particles)        # (p, n)
    all_obs_scores = batched_obs_score_fn(particles)     # (p, n, d)
    all_p = pt.exp(all_obs_logp)                         # (p, n) likelihoods

    # Mixture density for particle j: mean over l != j
    total_p = all_p.sum(axis=0, keepdims=True)          # (1, n)
    p_mix   = (total_p - all_p) / (particles.shape[0] - 1) # (p, n)

    # W_j = (1/n) * sum_i [ (p_j(x_i)/p_mix_j(x_i)) * score_j(x_i) ]
    ratio = all_p / p_mix                                # (p, n)
    return (ratio[:, :, None] * all_obs_scores).mean(axis=1) # (p, d)
```

Listing 3: Log-score WGF gradient (pseudocode).

MMD-score Wasserstein Gradient

For the squared-MMD score with kernel k, Lemma 2 of McLatchie et al. (2026) gives the decomposition $S(P_Q, x) = \int L(\theta_{1:2}, x)\, \mathrm{d}Q^{\otimes 2}(\theta_{1:2})$ with $L(\theta_1, \theta_2, x) = S(P_{\theta_1}, x) - \frac{1}{2}\|\mu_{\theta_1} - \mu_{\theta_2}\|_{\mathcal{H}}^2$. The discrete gradient is

$$\mathcal{W}_{\mathrm{MMD}}^{(j)} = \frac{1}{n}\sum_{i=1}^{n} \nabla_\vartheta S(P_{\vartheta^{(j)}}, x_i) - \frac{1}{p-1}\sum_{\ell \neq j} \nabla_\vartheta \tfrac{1}{2}\|\mu_{\vartheta^{(j)}} - \mu_{\vartheta^{(\ell)}}\|_{\mathcal{H}}^2 . \tag{13}$$

Both gradient terms require differentiating through the predictive distribution P_ϑ, which is possible only when P_ϑ is a continuous location-scale family (reparameterisation trick: $Y = \mu_\vartheta + \sigma_\vartheta \epsilon$, $\epsilon \sim \mathcal{N}(0,1)$). `MMDScore` is therefore restricted to continuous-response models; models with discrete likelihoods (Binomial, Poisson) use `LogScore`, which only requires `pytensor.grad` of the log-likelihood.

Euler–Maruyama Step (`particles.py`)

```
import numpy as np
import pytensor.tensor as pt

def em_step(particles, prior_logp_grad_fn, wgf_grad_fn,
            step_size, rng, lambda_n):
```

```
    """
    particles : np.ndarray  (p, d)   -- current particle positions
    Returns updated particles of the same shape.
    """
    # Prior gradient: nabla_theta log pi(theta)  --  shape (p, d)
    prior_grads = np.stack([prior_logp_grad_fn(particles[j])
                            for j in range(len(particles))])
    # WGF interaction term  --  shape (p, d)
    wgf_grads = wgf_grad_fn(particles)               # calls scoring.py

    drift  = lambda_n * wgf_grads - prior_grads
    noise  = np.sqrt(2.0 * step_size) * rng.standard_normal(particles.shape)
    return particles - step_size * drift + noise
```

Listing 4: One Euler-Maruyama step.

Main Sampler and API (`sampler.py`, `api.py`)

The public entry point mirrors `pm.sample_smc` in its signature and returns an ArviZ `DataTree` object (ArviZ 1.0), enabling seamless use of the existing diagnostic and plotting ecosystem:

```
def sample_pro(
    model=None,
    scoring_rule: str | ScoringRule = "log",
  n_particles: int = 128,
  n_steps: int = 2000,
  burn_in: int = 400,
    thinning: int = 1,
    step_size: float = 1e-3,
    learning_rate: float | str = "auto",
    kernel: str = "rbf",               # for MMD only
    random_seed=None,
    idata_kwargs: dict | None = None,
    progressbar: bool = True,
) -> xr.DataTree:
    """
    Sample from the Predictively Oriented (PrO) posterior.

    Uses mean-field Langevin dynamics (Wasserstein gradient flow) to
    evolve p interacting particles to approximate Q_n from Eq. (1).
    """
```

Listing 5: Public API signature.

These defaults are intended as practical starting points for typical laptops; paper-scale runs (e.g., larger p and K) remain available for final analyses.

The simulation loop, written to be backend-agnostic via NumPy with optional PyTensor compilation to JAX or Numba:

```
def _simulate(model, scoring_rule, params):
    prior_grad = compile_prior_gradient(model)   # PyTensor -> callable
    wgf_grad   = scoring_rule.compile_wgf(model) # PyTensor -> callable

    particles  = initialise_from_prior(model, params.n_particles)
    retained   = []

    with ProgressBar(params.n_steps) as bar:
        for k in range(params.n_steps):
            particles = em_step(
                particles, prior_grad, wgf_grad,
                params.step_size, rng, params.lambda_n
            )
            if k >= params.burn_in and k % params.thinning == 0:
```

```
                retained.append(particles.copy())
            bar.update()

    # retained: list of (p, d) arrays -> stack -> (draws, d)
    # reshape into chain/draw structure for DataTree
    return np.stack(retained)
```

Listing 6: Core simulation loop.

PyTensor Integration Details

The implementation exploits three PyTensor features that I have already worked with through my prior contributions:

1. `pytensor.grad` for $\nabla_\vartheta \log \Pi(\vartheta)$ (prior score) and $\nabla_\vartheta \log p_\vartheta(x)$ (likelihood score). Both are obtained through PyMC's `model.compile_logp()` graph, avoiding any manual differentiation.
2. `pytensor.vectorize` / `pt.vectorize` for batching the score computation over all p particles simultaneously, replacing a Python loop.
3. `pytensor.function(..., mode="JAX")` or `mode="NUMBA"` to compile the inner gradient loop, enabling GPU-accelerated runs.

The prior gradient graph is constructed once at the start of `sample_pro` and reused at every iteration, matching the pattern used in `pymc/variational/operators.py`. The WGF gradient is implemented via `pytensor.grad` and `pt.vectorize`; if `pt.vectorize` proves insufficiently mature during development, an equivalent `pt.scan` implementation serves as a tested fallback (cf. pytensor#1861, where I worked with `scan/op.py` validation directly).

ArviZ `DataTree` Output

The retained particle cloud from (11) is packaged as an ArviZ `DataTree` object. The p particles at each retained step are treated as p posterior draws per chain, following the standard ArviZ data schema. PrO-specific convergence quantities (particle spread $\sigma_k = \mathrm{std}(\{\vartheta_k^{(j)}\}_j)$ per step, mean score trajectory, and adaptive λ_n history) are stored inside the standard `sample_stats` group, ensuring compatibility with ArviZ `DataTree`/`InferenceData` conventions. Custom visualisations for these PrO-specific quantities are a stretch goal (D10); they may be contributed to the ArviZ project directly or added in `bambi-pro`, depending on mentor direction.

Deliverables

All deliverables are **required** unless marked *(stretch)*.

D1. Scoring Rule Abstraction. Abstract base class `ScoringRule` and concrete implementations `LogScore` and `MMDScore`, with PyTensor-compiled Wasserstein gradient functions. Both WGF gradients are fully derived in McLatchie et al. (2026); additional scoring rules are left to the stretch goals, avoiding scope creep from edge-case WGF derivations. Unit tests verifying correctness against analytical gradients on Gaussian models.

D2. Prior Gradient Infrastructure. Utility `compile_prior_gradient(model)` that wraps `model.compile_logp()` and `pytensor.grad` to return a callable $\nabla_\vartheta \log \Pi(\vartheta)$, respecting PyMC's value-variable and transform conventions.

D3. Euler–Maruyama Particle Stepper. Vectorised, numerically stable implementation of (10) with step-size schedule, gradient clipping, and particle-spread monitoring for stability. Initial deliverables target low-dimensional models; known performance limits for high-dimensional problems will be flagged clearly in the documentation. Support for both NumPy (development) and compiled backends.

D4. Main Sampler: `sample_pro`. Full simulation loop with burn-in, thinning, automatic λ_n, runtime warnings for particle collapse, and optional progress bar via `pymc.progress_bar`. Includes a bounded thin-wrapper milestone for `bambinos/bambi-pro`: translating a fitted Bambi model to `sample_pro`'s engine interface and returning ArviZ-compatible output for at least one canonical Bambi example.

D5. ArviZ `DataTree` Integration. Output to an ArviZ `DataTree` (ArviZ 1.0) with `posterior`, `log_likelihood`, `observed_data`, and `sample_stats` groups. The `sample_stats` group records PrO-specific quantities (particle spread σ_k, mean score trajectory, λ_n history) following ArviZ schema conventions. Data output is fully compatible with ArviZ `DataTree/InferenceData` structures.

D6. Comprehensive Test Suite. Unit tests for each sub-module; integration tests on: (a) a univariate Gaussian (ground truth available analytically), (b) logistic regression under correct specification vs. misspecification (binary classification problem from the paper), (c) a GMM clustering task (verifying non-collapse under misspecification). Regression tests pinning behaviour on paper's golf-putting dataset. Includes a runtime/memory benchmark suite reporting scaling over (p, n, d) for both `LogScore` and `MMDScore`, with documented recommended operating ranges.

D7. Tutorial Notebook. A fully executable Jupyter notebook demonstrating: (i) well-specified Gaussian: $Q_n \approx Q_n^\dagger$; (ii) logistic regression on misspecified binary labels, reproduced from the paper; (iii) Palmer penguins clustering with MMD score; (iv) comparison of `pm.sample` (NUTS) vs. `pm.sample_pro` via ArviZ plots.

D8. *(Stretch)* **Mini-batch / Streaming Data.** Adapt the WGF gradient to use data subsampling, enabling application to large datasets consistent with PyMC's existing Minibatch API (Hoffman & Blei, 2013). For MMD, evaluate approximate interaction variants (subsampled particle pairs or low-rank kernel surrogates) to reduce naive $O(p^2 n)$ cost.

D9. *(Stretch)* **Additional Scoring Rules.** CRPS (for continuous one-dimensional outputs) and energy score (multivariate), both of which satisfy the kernel decomposition of Lemma 2 in McLatchie et al. (2026) and so plug directly into the existing WGF machinery.

D10. *(Stretch)* **PrO Diagnostic Visualisations.** Custom PrO diagnostic visualisations for particle-spread trajectories, score convergence, and λ_n history; to be contributed upstream to ArviZ or housed in `bambi-pro` per mentor direction.

Project Timeline

GSoC 2026 standard coding period: **25 May – 24 August 2026** (12 weeks). The project is sized as **Large** (≈350 coding hours). The 12-week standard period accommodates 300 h of deliverable work; the remaining 50 h (D7 + documentation, Weeks 14–15) are scheduled in the officially permitted extended period through **19 September 2026** (well within the GSoC extended deadline of 2 November 2026). Community Bonding (30 h) is preparatory time outside the coding budget, giving 380 h in total.

Community Bonding . . . . . **30 h**
1 May – 24 May. Study `pymc/smc/` and `pymc/variational/` internals in depth; implement a reference WGF sampler in pure NumPy; draft API spec in a GitHub Discussion; solicit community feedback on API ergonomics; resolve open questions with mentors.

Weeks 1–2 . . . . . **40 h**
25 May – 7 Jun. **D2** Prior gradient infrastructure. Implement `compile_prior_gradient`; handle

unconstrained transforms (`model.rvs_to_transforms`) so particles live in unconstrained space; write unit tests.

Weeks 3–4 ..... **50 h**
8–21 Jun. **D1 (part 1)** Log-score WGF. Implement `LogScore` including (12) via `pytensor.grad` and `pt.vectorize`; validate against the analytical Gaussian gradient; write tests; mid-term check-in with mentors.

Weeks 5–6 ..... **50 h**
22 Jun – 5 Jul. **D3, D4 (draft)** Euler–Maruyama stepper and skeleton of `sample_pro`; run end-to-end on Gaussian model; confirm particle distributions converge to $\mathcal{N}(\mu, \Sigma)$. First working demo shared with mentors.

Week 7: Midterm Evaluation ..... **20 h**
6–12 Jul. Official GSoC midterm deadline (**10 Jul**). Deliverables D1 (log-score), D2, D3 complete and tested.

Weeks 8–9 ..... **50 h**
13–26 Jul. **D1 (part 2)** MMD-score WGF. Implement `MMDScore` including (13), RBF kernel and median-heuristic bandwidth, restricted to continuous-response models (reparameterisation trick required for the kernel-mean-embedding gradient); validate on Gaussian mixture; write tests.

Week 10 ..... **25 h**
27 Jul – 2 Aug. **D4** Complete `sample_pro`: auto λ_n, progress bar, stability checks; integration test on logistic regression (**15 h**). Begin `bambi-pro` thin wrapper milestone (**10 h**): Bambi-to-engine translation and first end-to-end check.

Week 11 ..... **25 h**
3–9 Aug. **D5** ArviZ DataTree integration. Map particle cloud to ArviZ `DataTree`; store PrO-specific quantities (particle spread, score trajectory, λ_n history) in `sample_stats`; verify schema-compatible ArviZ output; add timing/memory instrumentation hooks used by benchmark runs.

Weeks 12–13 ..... **40 h**
10–23 Aug. **D6** Full test suite. Integration tests (a)–(c); regression tests on golf-putting data; CI coverage targets; benchmark matrix over particle count/data size/model dimension and a short performance report with recommended defaults and cost envelopes. **Final work product submitted by 24 Aug** (official standard GSoC deadline).

Week 14 (extended period) ..... **30 h**
24–30 Aug. **D7** Tutorial notebook and `bambi-pro` polish. All four examples; ArviZ comparison plots; hosted in `pymc-pro` docs (**20 h**), plus wrapper hardening/documentation and one Bambi tutorial example in `bambi-pro` (**10 h**).

Week 15 (extended period) ..... **20 h**
31 Aug – 19 Sep. Docstrings and API reference page in Sphinx; address reviewer feedback; publish runtime-tuning guidance in docs; finalise stretch goals if time permits; final evaluation (extended coding period closes 2 Nov 2026 per official GSoC rules).

Availability. My final examination is **3 May 2026**; I have no academic commitments in June–August 2026. I can commit ≈ 35 hours/week during the core coding period.

Prior Contributions to PyMC and PyTensor

I have been contributing to PyMC and PyTensor since December 2025. All pull requests and their individual descriptions are listed below. Four PRs have been merged; five are under active review. My contributions span PyMC's logprob and sampling-statistics infrastructure, plus PyTensor's Scan internals, building the graph manipulation fluency that the PRO sampler's prior-gradient computation (`pytensor.grad` over `model.compile_logp()`) and particle infrastructure require.

Merged

- **pymc#7995** — *Logprob for leaky-ReLU switch transforms.* Created `pymc/logprob/switch.py` (305 lines). Recognises `switch(x > 0, x, a*x)` as a non-overlapping bijection and gates between the two branches' existing logp rules evaluated at the observed value, validating the positive-scale constraint at runtime. Refactored into an extensible module per reviewer request. Reviewed and merged by `ricardoV94`.

- **pymc#8008** — *Constant branches in switch, join, and make_vector logp.* Extended `find_measurable_stacks` and related rewrite rules in `logprob/mixture.py` and `logprob/tensor.py` to wrap compile-time constants as `dirac_delta` distributions, enabling exact logp when one branch is a fixed value. Closes #7711. Merged by `ricardoV94`.

- **pymc#8067** — *Logprob of* `observe(sum(NormalRV))`. Added a measurable graph rewrite in `logprob/arithmetic.py` that transforms `observe`$(\sum_i \mathcal{N}(\mu_i, \sigma_i))$ into a single equivalent Normal RV by closure under addition, enabling exact logp without sampling. Reviewed and merged by `ricardoV94`.

- **pymc#8015** — *Remove* ***`tune`*** *stat; fix non-discarding of warm-up draws.* Removes the legacy `tune` flag from all step-method `stats` dictionaries; introduces an `in_warmup` marker driven by the sampling loop.

Open / Under Review

- **pymc#8074** — *Elementwise add/sub of Normal RVs.* Extends #8067 to `pt.add` and `pt.sub` of arbitrary Normal RVs using Gaussian closure under addition/subtraction. 149 additions to `logprob/arithmetic.py`.

- **pymc#8058** — *Non-overlapping switch logp: equivalent spellings.* Extends `logprob/switch.py` to accept the swapped form `switch(cond, a*x, x)` and equivalent predicates (`x < 0`, `0 > x`, etc.) by canonicalising all matched forms before rewriting. Closes #8049.

- **pymc#8070** — *Multi-output IfElse logp for non-lazy backends.* Fixed a correctness bug in `logprob/mixture.py`: multi-output `IfElse` nodes were split into single-output nodes, breaking

cross-output dependencies and triggering spurious `SpecifyShape` assertions from dead branches under non-lazy evaluation.

- **pymc#8016** — *Scan logprob when observed adds broadcastable axes.* Regression test for a crash in the measurable Scan rewrite when the observed tensor is more broadcastable than the generative graph (e.g. shape `(t, 1)` vs. `(t, n)`). Fix delegated to pytensor#1861.
- **pytensor#1861** — *Relax Scan broadcastability checks.* Replaces rigid equality checks in `scan/op.py` with `type_input.filter_variable(type_output)` compatibility, allowing inner outputs to be strictly more specific than `outputs_info` without spurious validation errors. Directly unblocks pymc#8016.

Collectively these contributions demonstrate hands-on fluency with: (a) PyTensor graph rewrites and the Op/Apply/rewrite-rule abstraction; (b) PyMC's logprob inference pass and value-variable mechanics; (c) Scan and IfElse measurable graph handling; (d) sampling-loop internals, CI workflows, and project contribution norms. (a)–(c) bear directly on the PrO implementation; (d) is required to land it in the codebase.

Engagement with the Organisation

I have:

- Had multiple code-review exchanges with `ricardoV94` on the PyMC core team across the PRs listed above.
- Opened follow-up issues (#8073, #8052, #8051, #8050, #8049) demonstrating continued investment beyond individual PRs.
- Familiarised myself with PyMC Discourse and the organisation's contribution and communication norms.

Related Work

PrO posteriors extend the generalised Bayes framework of Bissiri et al. (2016); Knoblauch et al. (2019) and the MMD-based posteriors of Chérief-Abdellatif & Alquier (2020); Matsubara et al. (2022). Their closest computational predecessor is Bickford Smith et al. (2023), which derives an asymptotically exact mean-field Langevin sampler for the squared-MMD special case; the general theory covering arbitrary proper scoring rules and all misspecification regimes is due to McLatchie et al. (2026).

No existing probabilistic programming library implements PrO posteriors. SVGD (Liu & Wang, 2016) and Stein operators in `pymc/variational/` share the particle-system architecture but optimise a different objective (the KL between Q and the posterior). Pathfinder (Zhang et al., 2022), recently added to PyMC, provides a fast Laplace-like approximation; `sample_pro` is complementary, targeting robustness to misspecification at the cost of more computation.

References

Bissiri, P.G., Holmes, C.C., Walker, S.G. (2016). A general framework for updating belief distributions. *J. Royal Stat. Soc. B*, 78(5), 1103–1130.

Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Chérief-Abdellatif, B.-E., Alquier, P. (2020). MMD-Bayes: Robust Bayesian estimation via maximum mean discrepancy. *Proc. AISTATS*.

Gelman, A., Carlin, J., Stern, H., Dunson, D., Vehtari, A., Rubin, D. (2013). *Bayesian Data Analysis*, 3rd ed. CRC Press.

Gneiting, T., Raftery, A.E. (2007). Strictly proper scoring rules, prediction, and estimation. *J. Am. Stat. Assoc.*, 102, 359–378.

Hoffman, M.D., Blei, D.M., Wang, C., Paisley, J. (2013). Stochastic variational inference. *J. Mach. Learn. Res.*, 14, 1303–1347.

Jordan, R., Kinderlehrer, D., Otto, F. (1998). The variational formulation of the Fokker–Planck equation. *SIAM J. Math. Anal.*, 29(1), 1–17.

Knoblauch, J., Jewson, J., Damoulas, T. (2019). Generalized Variational Inference: Three arguments for deriving new Posteriors. *arXiv:1904.02063*.

Liu, Q., Wang, D. (2016). Stein variational gradient descent. *Advances in NeurIPS*.

Matsubara, T., Knoblauch, J., Briol, F.-X., Oates, C.J. (2022). Robust generalised Bayesian inference for intractable likelihoods. *J. Royal Stat. Soc. B*, 84(3), 997–1022.

McLatchie, Y., Chérief-Abdellatif, B.-E., Frazier, D.T., Knoblauch, J. (2026). Predictively Oriented Posteriors. *arXiv:2510.01915v2*.

Bickford Smith, F., Kirsch, A., Farquhar, S., Gal, Y., Foster, A., Rainforth, T. (2023). Prediction-oriented Bayesian active learning. *arXiv:2304.08151*.

Wild, V. D., Ghalebikesabi, S., Sejdinovic, D., Knoblauch, J. (2023). A rigorous link between deep ensembles and (variational) Bayesian methods. *arXiv:2305.15027*.

Zhang, L., Carpenter, B., Gelman, A., Vehtari, A. (2022). Pathfinder: Parallel quasi-Newton variational inference. *J. Mach. Learn. Res.*, 23(306), 1–49.

AI Use Disclosure

Gemini and ChatGPT were used during the drafting of this proposal as a writing and editing assistant: checking grammar, suggesting phrasing alternatives, and reformatting LaTeX markup. All technical content (theorem statements, algorithm design, implementation plans, and code sketches) was authored and verified by me against the source paper (McLatchie et al., 2026) and the PyMC/PyTensor codebases. No AI tool generated any novel technical claim on my behalf.